

USACH

SISTEMA DE ADMINISTRACIÓN ACADÉMICA UNIVERSITARIA

Start

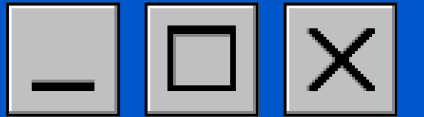
PROFESOR:
MATÍAS CALDERÓN



AYUDANTE:
FERNANDO SOLÍS



Integrantes



Inicio Desarrollo Final

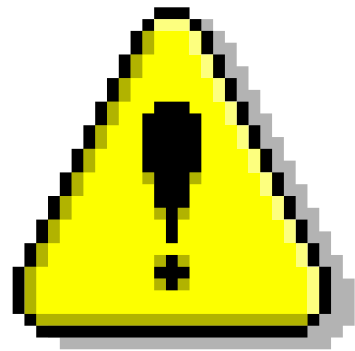
 I1: BENJAMÍN
ARAYA

 I2: VICENTE
TAPIA

 I3: DANIEL
DÍAZ

 I4: JORDAN
GONZÁLEZ

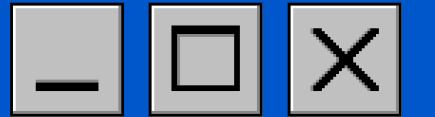




Una universidad requiere una intranet para **centralizar** y **automatizar** la gestión académica de sus **estudiantes**, reemplazando procesos manuales o dispersos. El sistema debe manejar información de **estudiantes**, **asignaturas**, **secciones**, **semestres** y **calificaciones**, garantizando que cada usuario como **profesores**, **estudiantes** y **administradores**, acceda únicamente a la información y funciones que le corresponden según su rol.



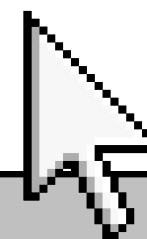
¿Qué debemos desarrollar?



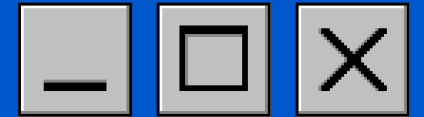
Inicio Desarrollo Final



- **CRUD** de **estudiantes y asignaturas** con autenticación JWT y control de roles.
- **SP de cierre de semestre:** congelar notas, calcular Promedio ponderado acumulado y actualizar estado del **estudiante**.
- **SP de inscripción en cursos con descuento de cupos en una sola transacción.**
- **Trigger** que bloquea inscripción si el **estudiante** no cumple los pre-requisitos.
- **Trigger** que impide ingresar notas fuera del calendario académico.
- **Endpoint** de malla curricular con estado por **asignatura**.
- **Reporte de reprobación histórica mediante vista materializada**, con alerta para cursos sobre el 40%.



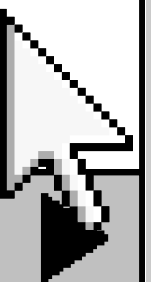
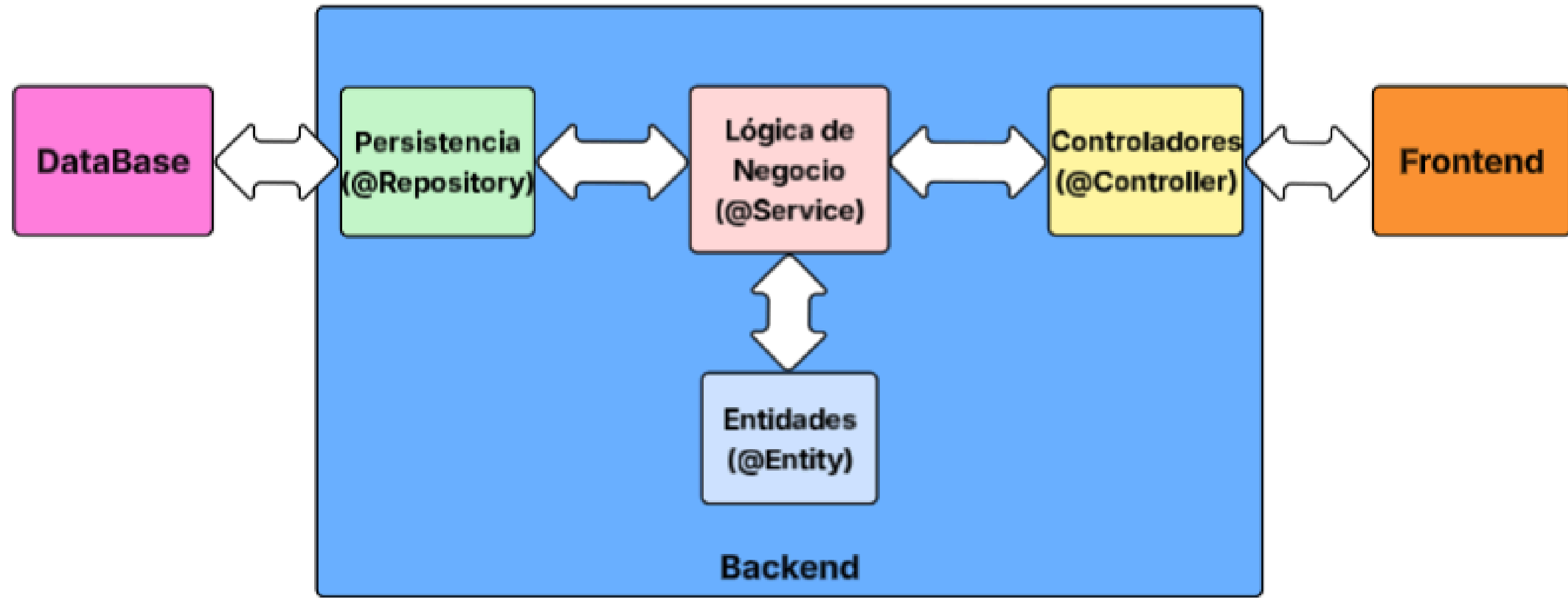
Arquitectura



Inicio **Desarrollo** Final



ARQUITECTURA POR
CAPAS – MONOLÍTICA

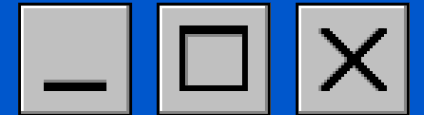




A diagram showing a rectangular button with a thick black border and a light gray fill. To the left of the button is a speech bubble containing a blue lowercase letter 'i'. A white mouse cursor with a black outline is positioned at the bottom right corner of the button.



Tecnologías Usadas



Inicio Desarrollo Final



MER

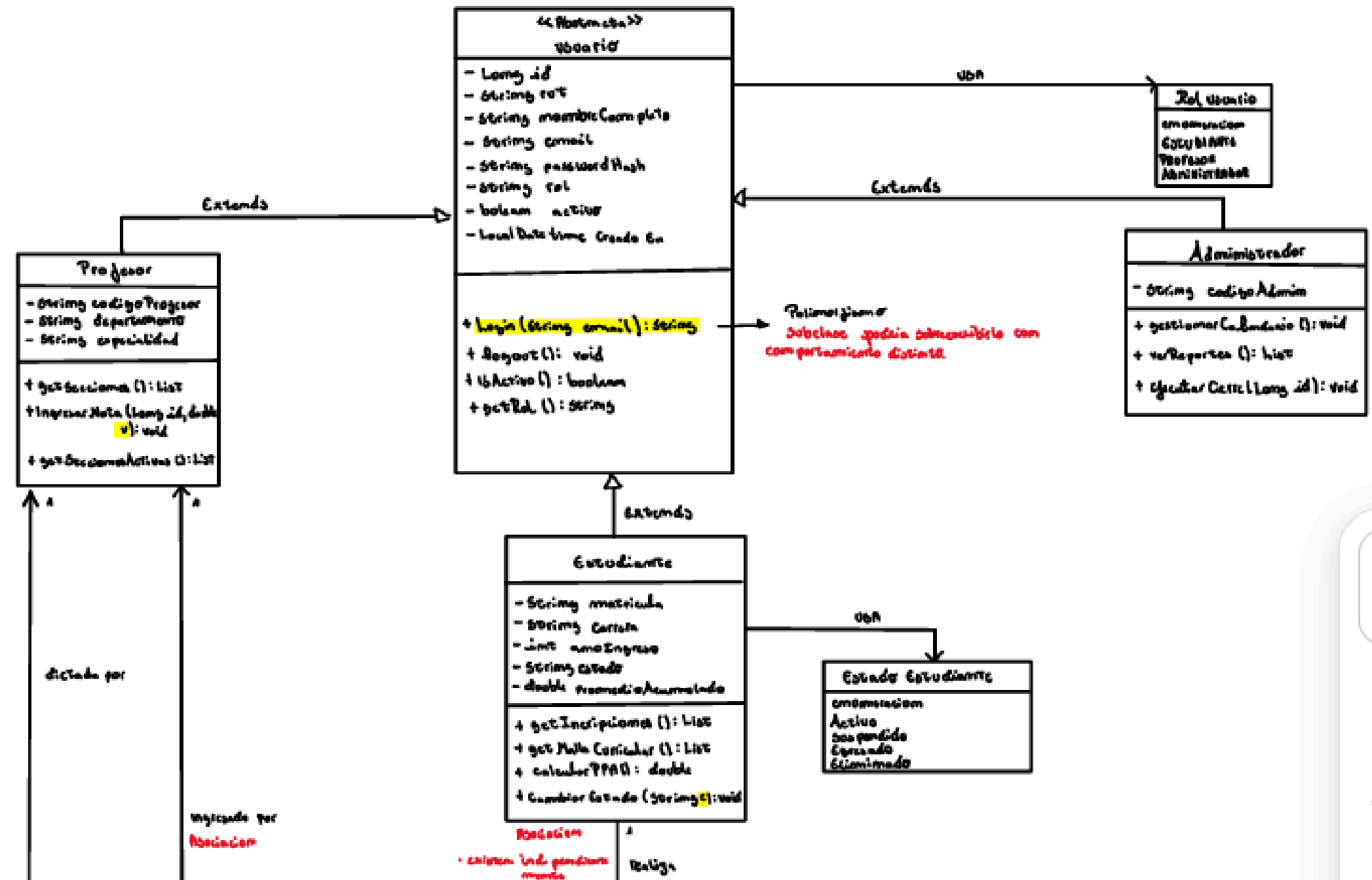


Diagrama de Modelado de Entidades y Relaciones (MER)

Clases y Atributos:

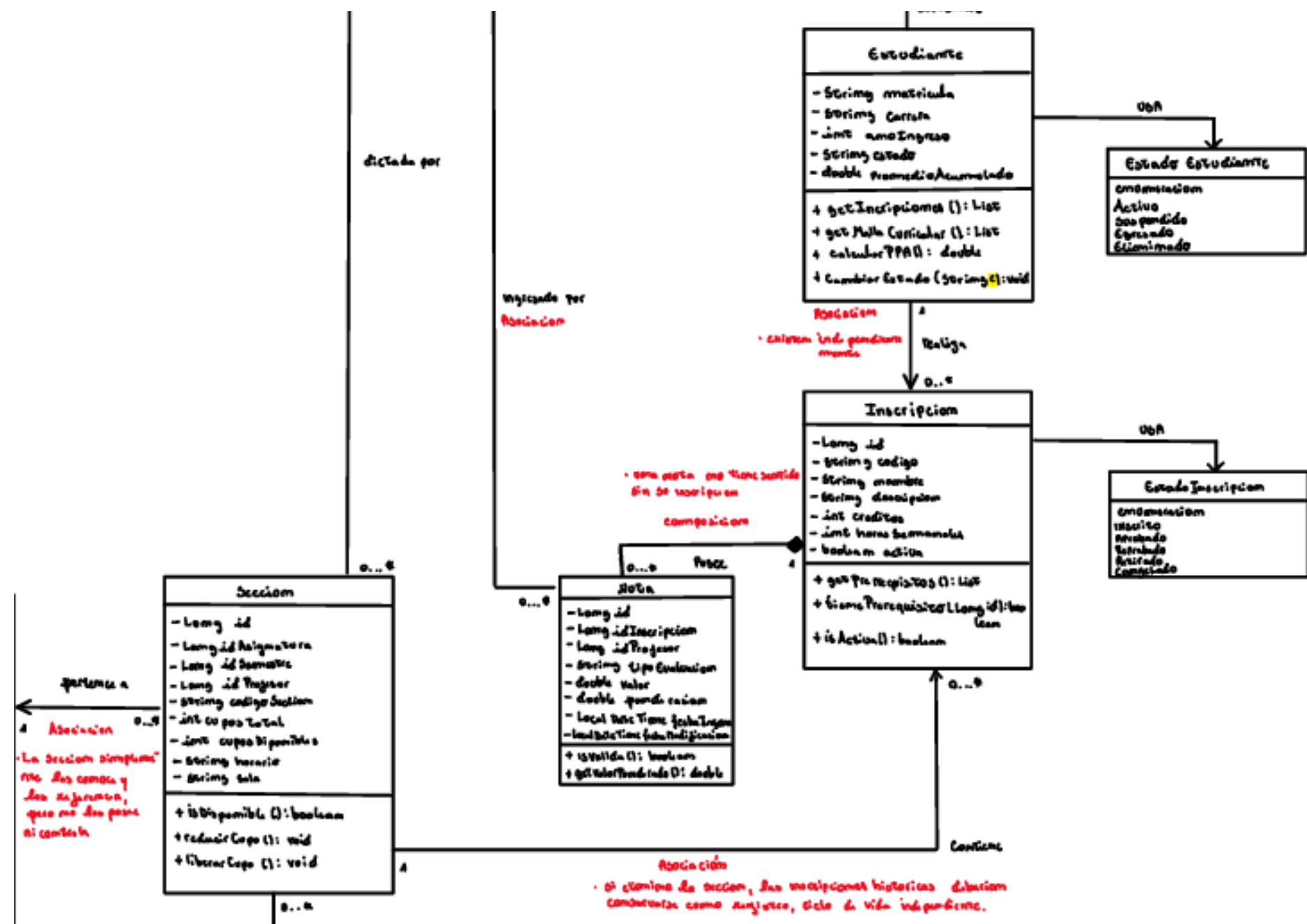
- Estudiante**
 - String matricula
 - String carrera
 - int añoIngreso
 - String estado
 - double promedioAcumulado
 - + getInscripciones(): List
 - + getNotaCursar(): List
 - + calcularPPA(): double
 - + cambiarEstado(String): void
- Sección**
 - Long id
 - Long idAsignatura
 - Long idSemestre
 - String codigoSeccion
 - int cupoTotal
 - int cupoDisponibles
 - String horario
 - String sala
 - + isDisponible(): boolean
 - + reducirCupo(): void
 - + liberarCupo(): void
- Inscripción**
 - Long id
 - String codigo
 - String nombre
 - String descripcion
 - int credits
 - int horasSemanales
 - boolean action
 - + getPrerequisitos(): List
 - + tienePrerequisito(Long id): boolean
 - + isAction(): boolean
- Profesor**
 - Long id
 - Long idInscripcion
 - Long idProfesor
 - String tipoExclusion
 - double valor
 - double quandoCausa
 - LocalDateTime fechaFin
 - LocalDateTime fechaInicio
 - + isValida(): boolean
 - + getValorQuandoCausa(): double
- Estado Estudiante**
 - enumeracion
 - Activo
 - Suspendido
 - Expulsado
 - Eliminado
- Estado Inscripción**
 - enumeracion
 - Inscrito
 - Retirado
 - Suspendido
 - Cancelado

Asociaciones:

- Estudiante a Inscripción (Asociación):** Multiplicidad 1 en Estudiante, 0..* en Inscripción. Rol: "Asociación". Nota: "cálculo del promedio acumulado".
- Inscripción a Sección (Asociación):** Multiplicidad 0..* en Inscripción, 1 en Sección. Rol: "Asociación". Nota: "si contiene la sección, las inscripciones históricas deberían comportarse como auxiliares, tipo de vida independiente".
- Inscripción a Profesor (Asociación):** Multiplicidad 0..* en Inscripción, 0..* en Profesor. Rol: "Asociación". Nota: "como parte de la inscripción sin de inscripción".
- Sección a Inscripción (Asociación):** Multiplicidad 1 en Sección, 0..* en Inscripción. Rol: "Asociación". Nota: "La Sección simplemente no los causa y los auxiliares, pero no los causa ni combata".

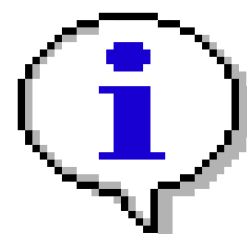
Estados:

- Estudiante:** Transición de "Estado Estudiante" (Activo, Suspendido, Expulsado, Eliminado).
- Inscripción:** Transición de "Estado Inscripción" (Inscrito, Retirado, Suspendido, Cancelado).

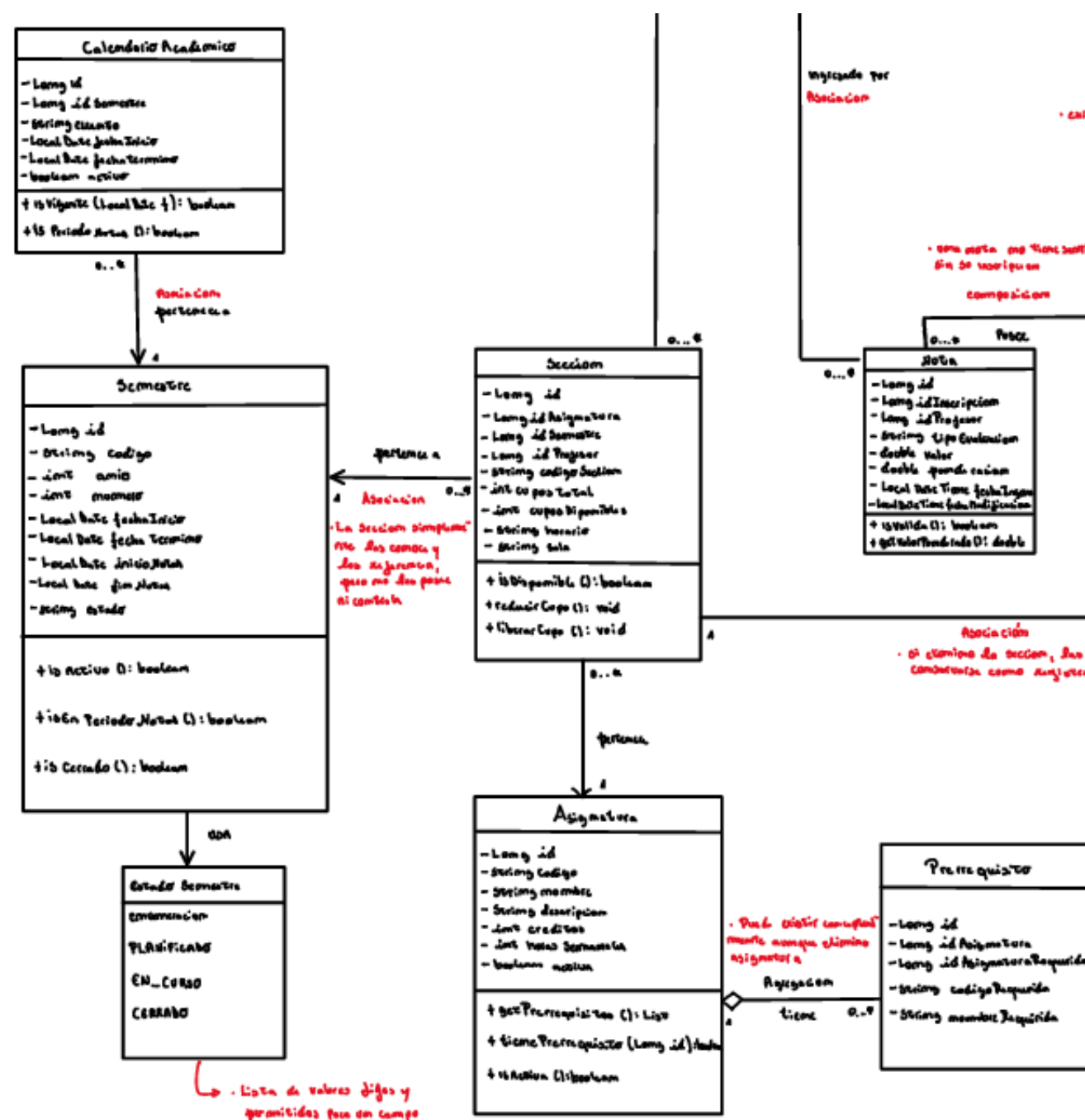


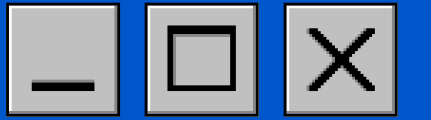
Tecnologías Usadas

Inicio Desarrollo Final



MER

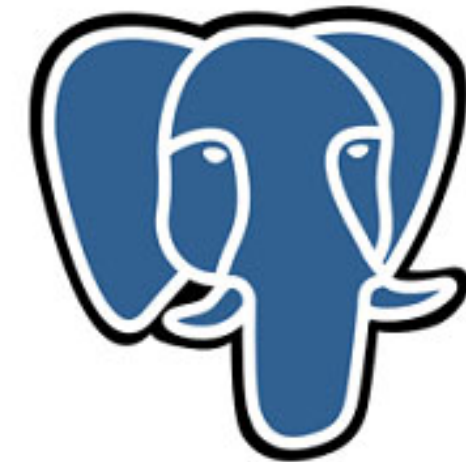




Inicio **Desarrollo** Final



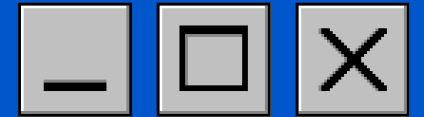
DATABASE



PostgreSQL



Tecnologías Usadas



Inicio **Desarrollo** Final



BACKEND



v4.0.6



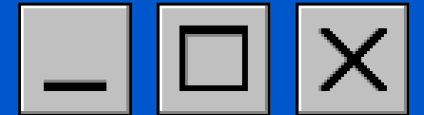
Dependencias usadas:

- spring-boot-starter-web
- spring-boot-starter-data-jdbc
- spring-boot-starter-security
- postgresql
- lombok
- springdoc-openapi (Swagger UI)
- spring-boot-starter-test
- jjwt-api / jjwt-impl / jjwt-jackson

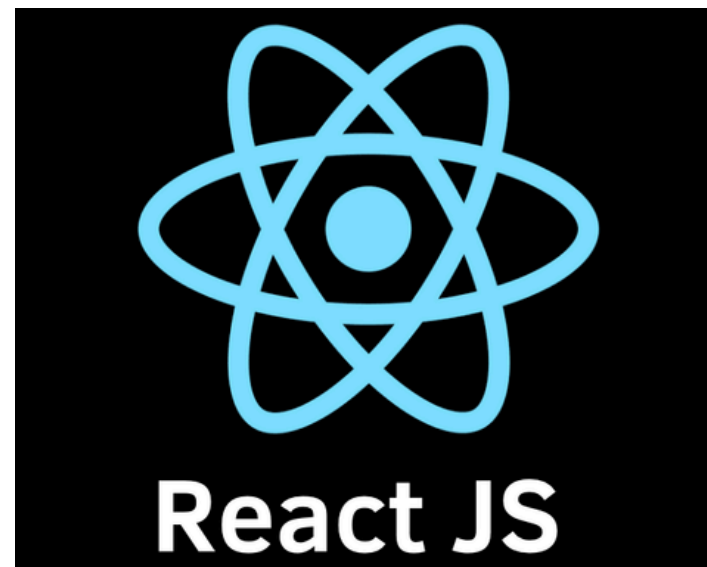


21 jdk





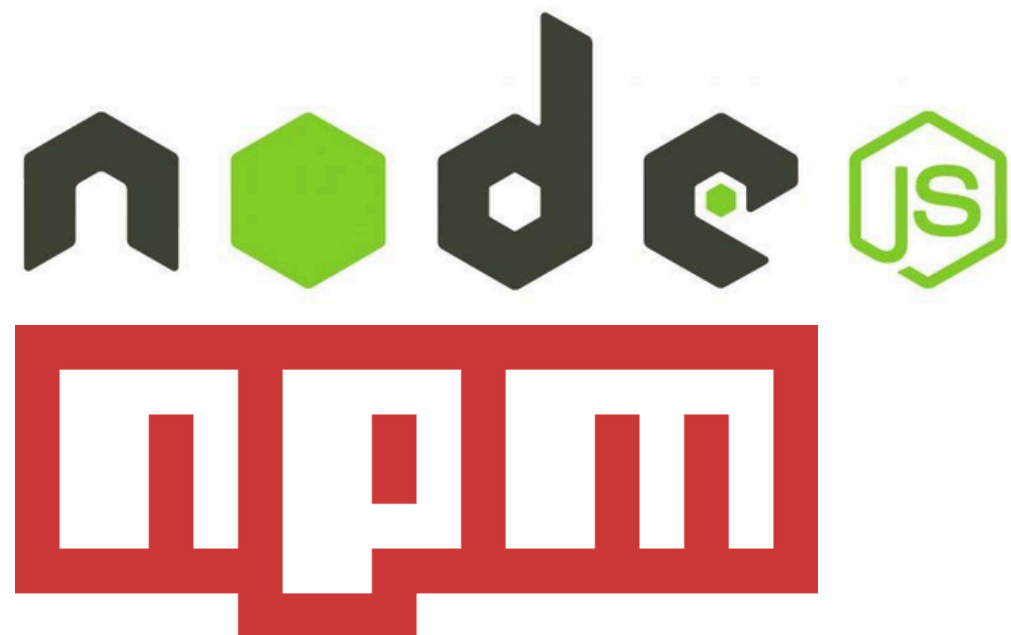
Inicio **Desarrollo** Final



◀◀◀
v18.x



FRONTEND

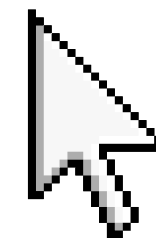


◀◀◀
v22.22.0

◀◀◀
v10.9.4

Librerías utilizadas en frontend:

- Bootstrap (estilo y diseño)
- Axios (HTTP → backend)
- React Router DOM (componentes)



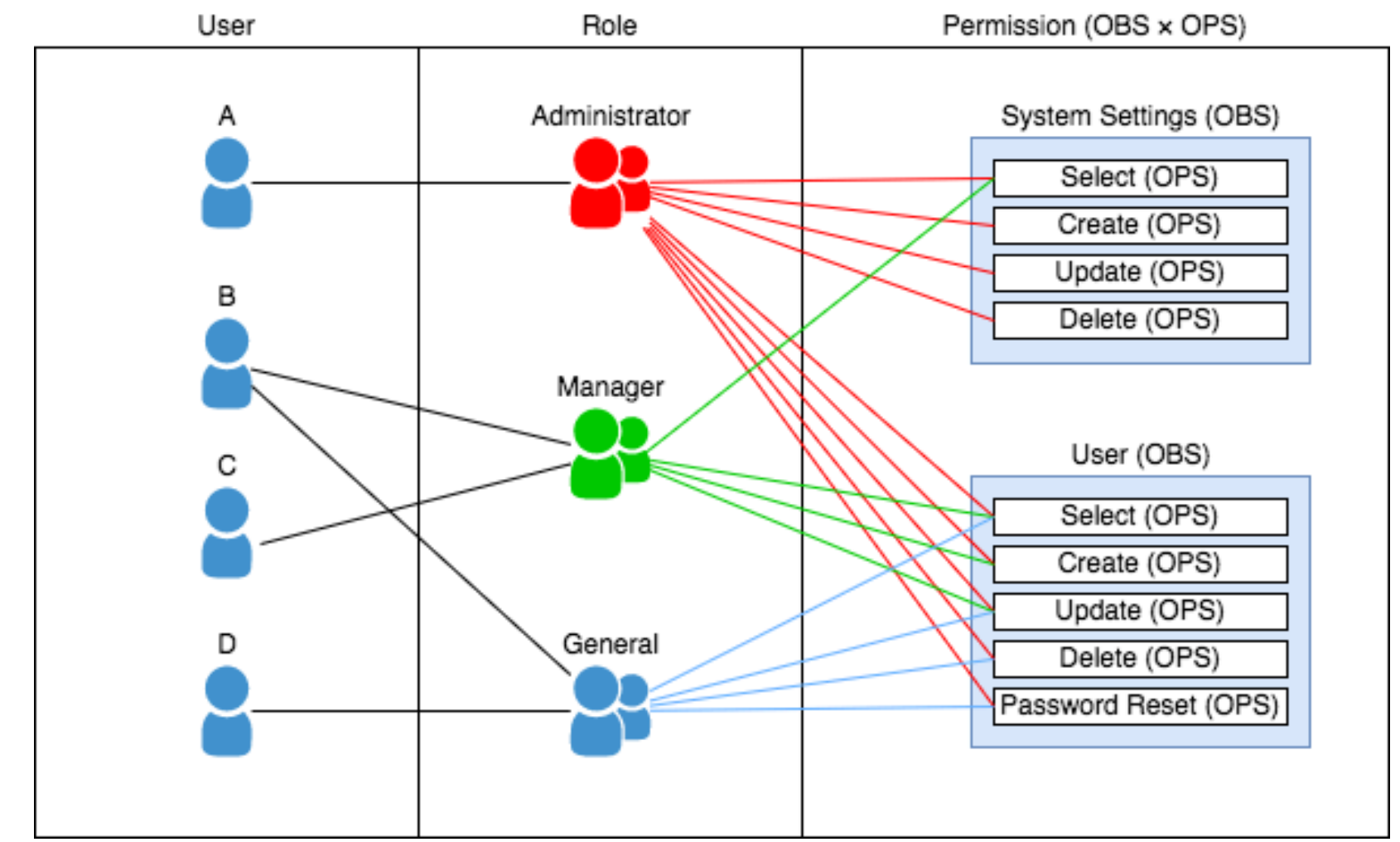


SEGURIDAD

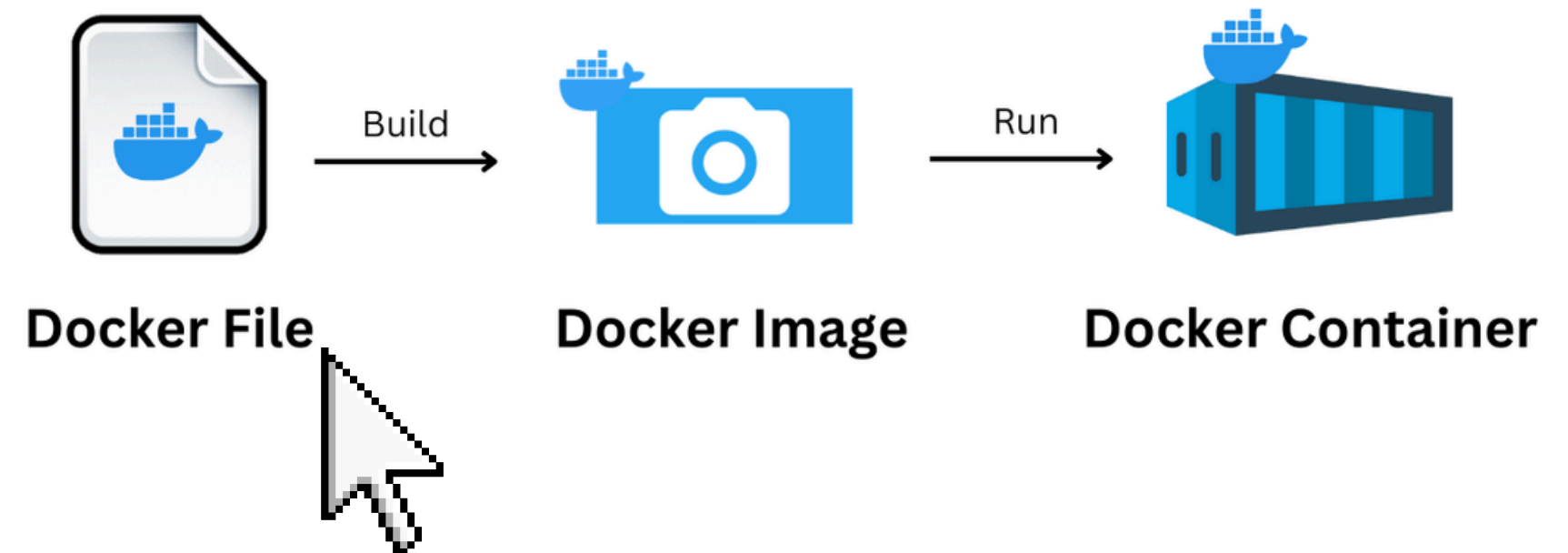
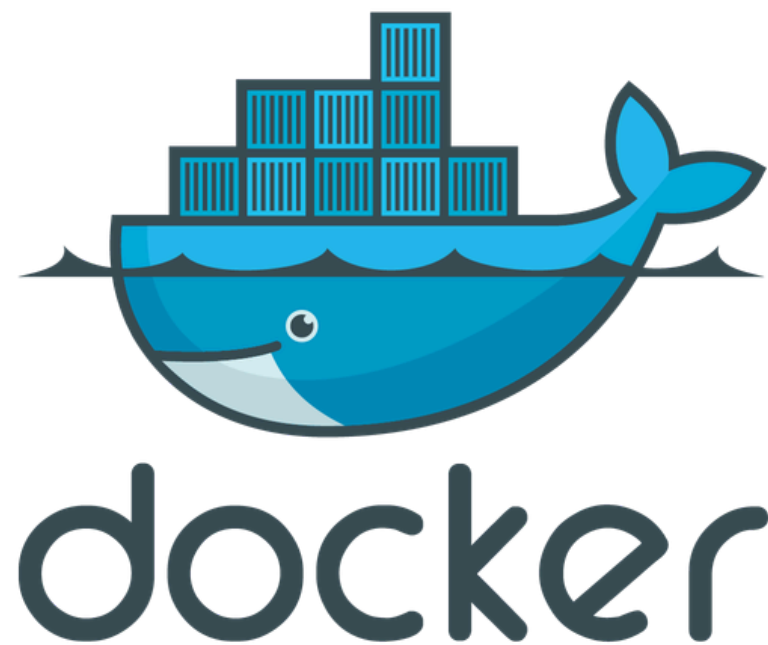
json web
token

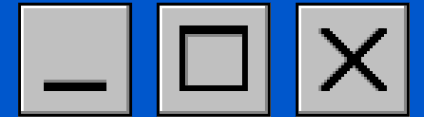


Role-Base Access Control (RBAC)

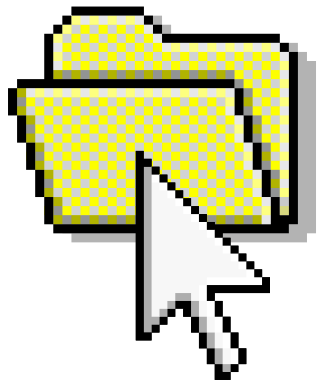


Inicio **Desarrollo** Final





Inicio Desarrollo Final



Active branches						
Branch		Updated	Check status	Behind	Ahead	Pull request
feature/modulo-usuario		16 hours ago		6	23	#4 ...
feature/modulo-estudiante		18 hours ago		6	8	#3 ...
feature/modulo-academico		yesterday		6	7	...
feature/modulo-profesor		2 days ago		6	10	#2 ...



**TRABAJO REALIZADO
POR RAMAS.**



Inicio **Desarrollo** Final

Integrante 1 – Fundación + Auth + Seguridad

Tablas BD: **usuario**

Archivos Spring Boot:

- pom.xml, application.properties, SecurityConfig.java, CorsConfig.java, JwtFilter.java, JwtUtil.java, AuthResponse.java, UsuarioEntity.java, UsuarioRepository.java, AuthService.java, AuthController.java

Endpoint clave: **POST /api/auth/login**

Prioridad: **Primero – todo depende de esto.**

Integrante 2 – Módulo Estudiante + Inscripción

Tablas BD: **estudiante, inscripcion**

Archivos Spring Boot:

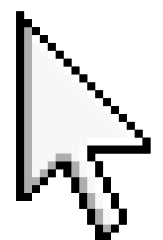
- EstudianteEntity.java, InscripcionEntity.java, EstudianteRepository.java, InscripcionRepository.java, EstudianteService.java, InscripcionService.java, EstudianteController.java, InscripcionController.java

Objetos BD avanzados: **SP sp_inscribir_alumno, Trigger check_prerequisitos, Índice en estudiante.matricula**

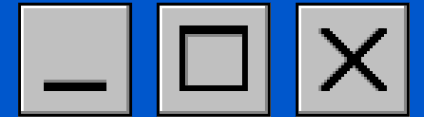
Endpoints clave: **GET/POST/PUT/DELETE /api/estudiantes, POST /api/inscripciones, GET /api/estudiantes/{id}/malla**



**DIVISIÓN PARA
DATABASE Y BACKEND**



Trabajo en equipo



Inicio **Desarrollo** Final

Integrante 3 – Módulo Académico

Tablas BD: carrera, asignatura, prerequisite, semestre, seccion

Archivos Spring Boot:

- AsignaturaEntity.java, CarreraEntity.java, SemestreEntity.java, SeccionEntity.java, AsignaturaService.java, SemestreService.java, AsignaturaController.java, SemestreController.java + repository por cada entity

Objetos BD avanzados: SP sp_cierre_semestre, Trigger check_calendario_notas, Índices en asignatura.codigo, seccion.semestre_id

Endpoints clave: GET/POST/PUT/DELETE /api/asignaturas, POST /api/semestres/{id}/cierre

Integrante 4 – Módulo Profesor + Notas + Reportes

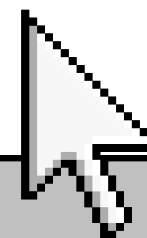
Tablas BD: profesor, nota, auditoria

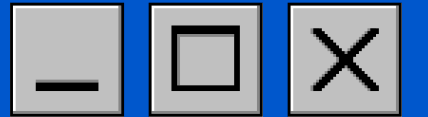
Archivos Spring Boot:

- ProfesorEntity.java, NotaEntity.java, AuditoriaEntity.java, ProfesorRepository.java, NotaRepository.java, ProfesorService.java, NotaService.java, ProfesorController.java, NotaController.java, ReporteController.java

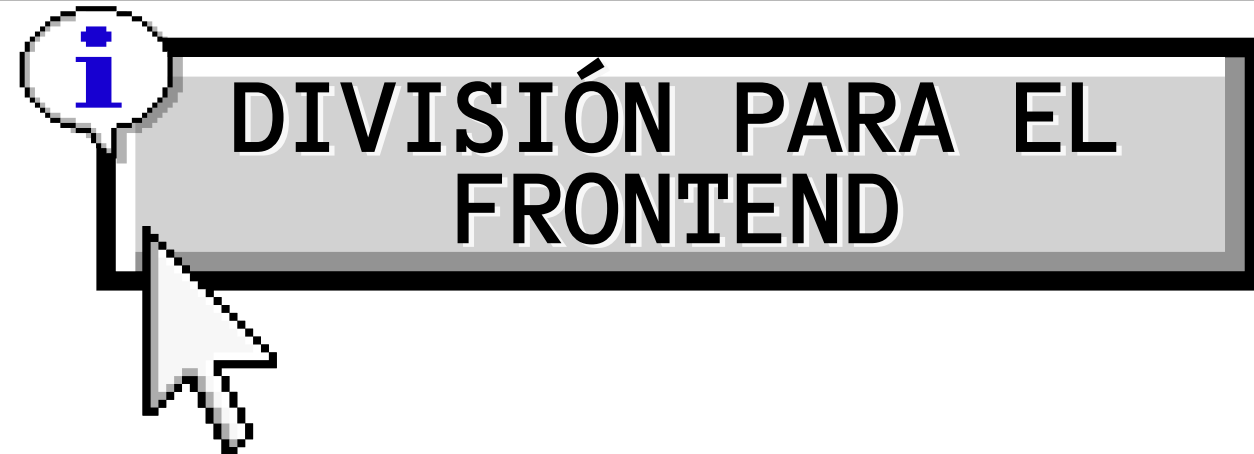
Objetos BD avanzados: Vista materializada mv_tasa_reprobacion, Función refresco MV

Endpoints clave: GET/POST/PUT/DELETE /api/profesores, POST /api/notas (solo PROFESOR), GET /api/notas (filtrado por rol), GET /api/reportes/reprobacion





Inicio Desarrollo Final



Integrante 1 – Login + Estructura base

Vistas: `LoginPage.jsx`, `NotFoundPage.jsx`

Responsabilidades extra:

- Crear estructura base del proyecto React
- Configurar Bootstrap
- Configurar Axios + AuthContext
- Subir estructura vacía para que todos partan

Endpoint que consume: `POST /api/auth/login`

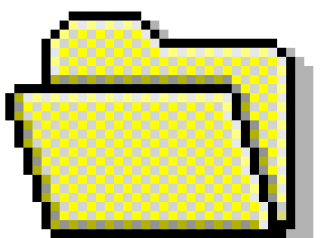
Prioridad: Primero – todo depende de esto.

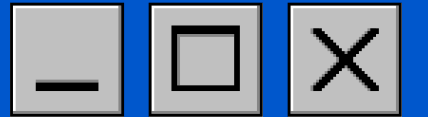
Integrante 2 – Dashboard estudiante

Vistas: `StudentDashboard.jsx`,
`StudentProfile.jsx`, `StudentCurriculum.jsx`,
`StudentEnrollments.jsx`, `EnrollForm.jsx`

Endpoints que consume:

- `GET /api/students/{id}`
- `GET /api/students/{id}/curriculum`
- `GET /api/enrollments/student/{id}`
- `POST /api/enrollments/enroll`





Inicio **Desarrollo** Final

Integrante 3 – Dashboard académico

Vistas: AdminDashboard.jsx, SubjectList.jsx, SubjectForm.jsx, SectionList.jsx, SemesterList.jsx, SemesterClose.jsx

Endpoints que consume:

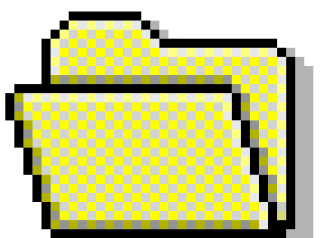
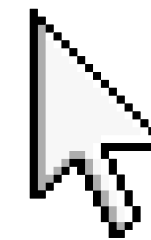
- GET/POST/PUT/DELETE /api/subjects
- GET/POST /api/sections
- POST /api/semesters/{id}/close

Integrante 4 – Dashboard profesor + reportes

Vistas: ProfessorDashboard.jsx, ProfessorCourses.jsx, GradeForm.jsx, StudentGrades.jsx, FailureReport.jsx

Endpoints que consume:

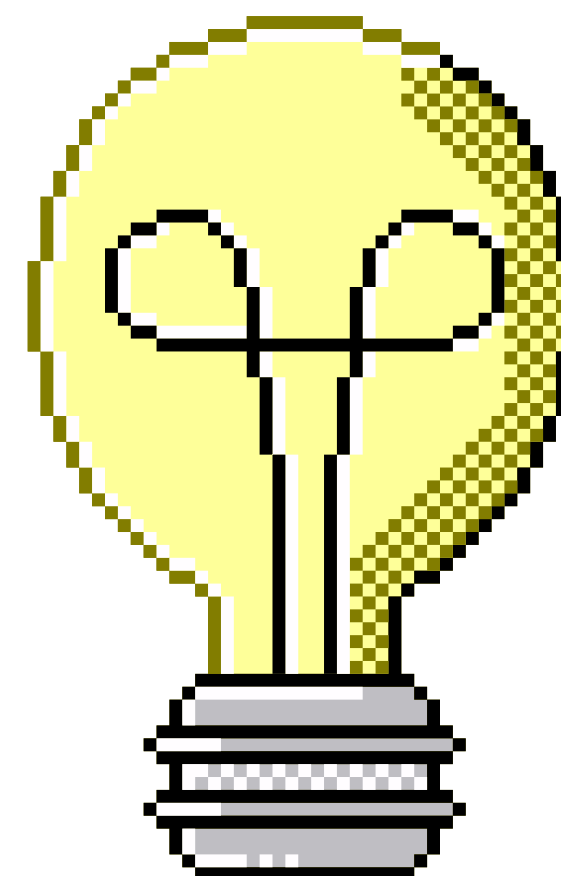
- GET /api/professors/{id}/sections
- POST /api/grades
- GET /api/grades
- GET /api/reports/failure-rate



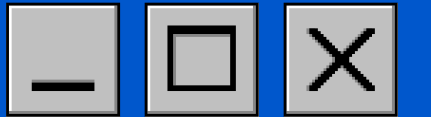


Inicio Desarrollo Final

HORA DE BOOTEAR EL
 PROYECTO



Final



Inicio Desarrollo **Final**

CONCLUSIÓN

No se logró completar al 100% el despliegue en Docker debido a cierta desorganización en el trabajo separado, ya que se utilizaron distintas formas de desarrollo entre los integrantes.

